Exclusive Self Attention

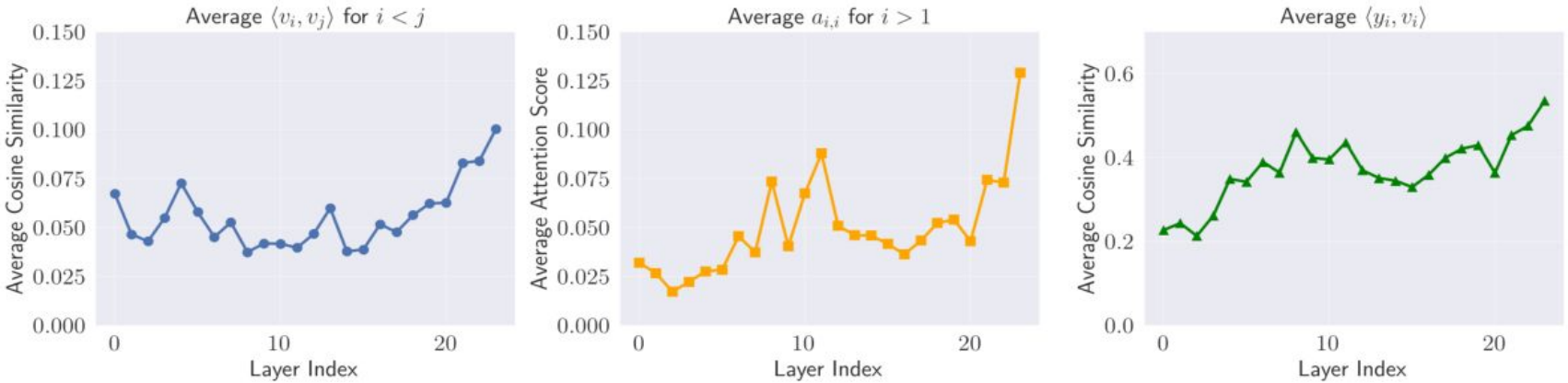
Интро

- Главная идея статьи в том, чтобы взять стандартный attention и спроецировать результат на подпространство
- Выход attention для каждого токена и каждой головы проецируется на подпространство, ортогональное его собственному value-вектору
-

Зачем это нужно

- В трансформере self-attention агрегирует информацию из контекста, а FFN преобразует представление каждого токена
- Автор обнаружил, что выход self-attention часто имеет высокое косинусное сходство с собственным value-вектором токена
- Автор называет это attention similarity bias. По гипотезе автора, attention частично дублирует позиционную обработку

Attention similarity bias: экспериментальное наблюдение



- Value-векторы разных токенов положительно коррелируют (слева)
- Внимание к собственной позиции относительно велико (по центру)
- $\cos(y_i, v_i)$ высок и в среднем растет с глубиной слоя (справа)

Как XSA решает эту проблему

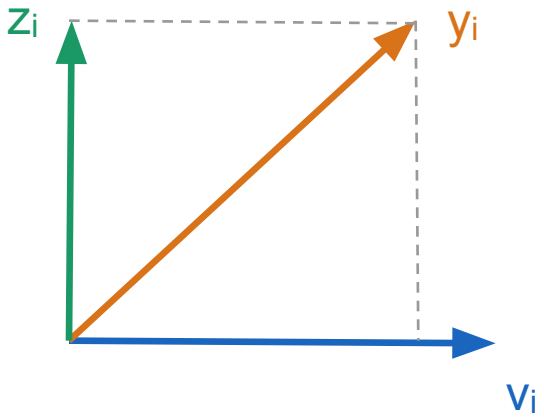
Обычный attention:

$y_i = \sum a_{ij} v_j \quad (j \le i)$

Вычитаем проекцию на собственный v_i:

$z_i = y_i - [(y_i^T v_i) / \|v_i\|^2] v_i$

Результат: $z_i^T v_i = 0$, если $v_i \neq 0$.



Для каждого токена в каждой голове внимания удаляем из выхода attention компоненту вдоль собственного value-вектора v_i. Это происходит до объединения голов и применения W_o. Веса внимания при этом не меняются

Реализация XSA на PyTorch

```
import torch.nn.functional as F

Y = F.scaled_dot_product_attention(
    Q, K, V, is_causal=True
)
Vn = F.normalize(V, dim=-1)
Z = Y - (Y * Vn).sum(dim=-1, keepdim=True) * Vn
```

Vn — нормализуем каждый value-вектор по последней размерности.
(Y * Vn).sum(...) – скалярная проекция Y на направление Vn.
Умножаем на Vn и вычитаем полученную векторную проекцию.

Q, K, V: [batch, heads, tokens, head_dim]. Отдельная проекция для каждой головы.
attention считается как обычно, меняется его выход

Экспериментальная постановка

NanoGPT + RoPE; LayerNorm после embeddings

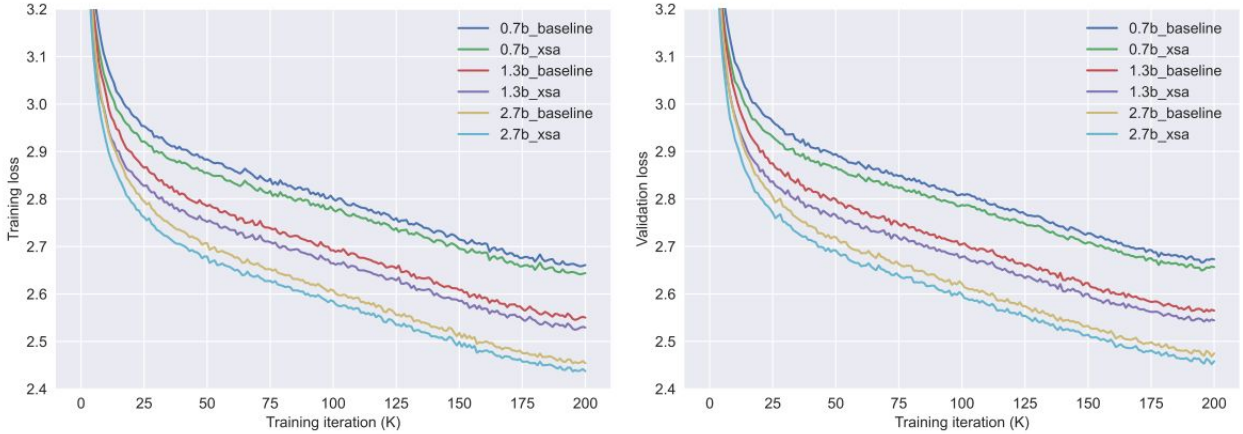
Модели: 0.7B / 1.4B / 2.7B параметров без embeddings

FineWeb-100BT; GPT-2 tokenizer; validation — 0.05% токенов

Контекст 2048; batch 256; 200K шагов ≈ 100B токенов

AdamW; warmup 2K; cosine decay. LR baseline используется и для XSA

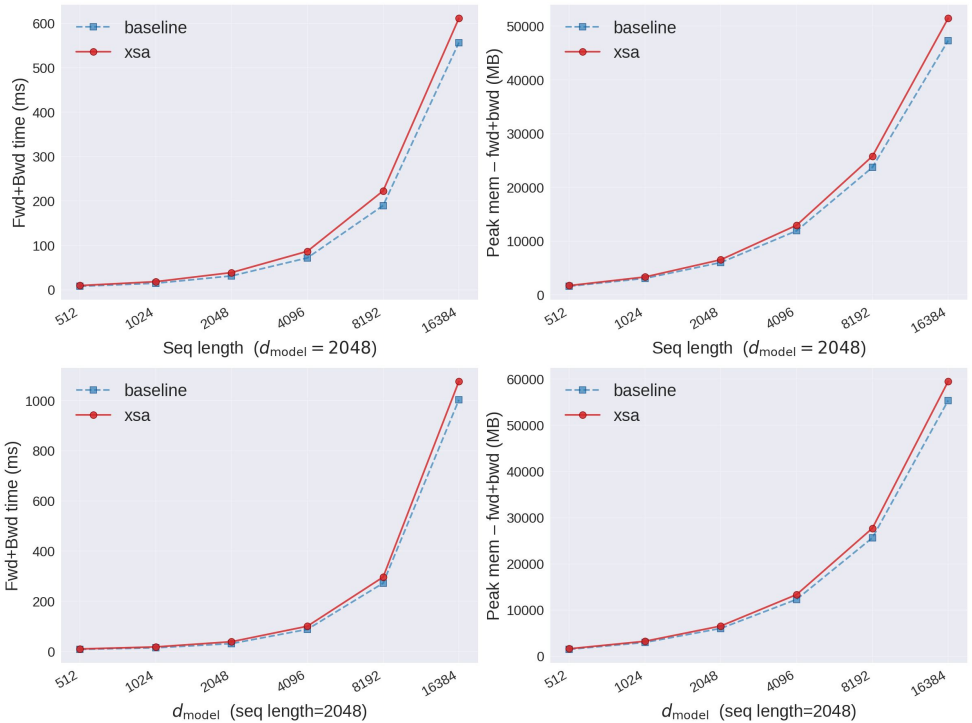
Качество: loss и downstream-задачи



Размер	**SA, среднее**	**XSA, среднее**	**Δ, п.п.**
0.7B	53.22	53.48	+0.26
1.3B	56.16	57.19	+1.03
2.7B	58.06	59.42	+1.36

Среднее по 8 задачам (accuracy / acc_norm); выигрыш не на каждой задаче.

Вычислительные затраты



Бенчмарк: attention + FFN

B200; BF16; batch 32

Forward + backward и пик памяти

Автор оценивает overhead на вычисление XSA как небольшой

Выводы и ограничения

- При использовании XSA ниже train/validation loss и выше средний результат на восьми тестовых задачах
- Разница увеличивается на длинных контекстах (512–16384)
- Проверяли на задаче language modeling с моделями размером до 2.7B
- Более крупные модели и другие модальности остались за рамками исследования